

P2066R0: Suggested draft TS for C++ Extensions for Transaction Memory Light

Introduction

This paper presents suggested wording for a future Technical Specification on a lighter variant of Transactional Memory. See P1875R0 "Transactional Memory Lite Support in C++" for discussion.

Acknowledgements

Thanks to Hans Boehm, Victor Luchangco, Michael Spear, and Michael Wong.

Wording changes

In 5.11 [lex.key], add **atomic** to table 5 [tab:lex:key].

Change in 6.9.2.1 [intro.races] paragraph 6:

Atomic blocks as well as ~~Certain~~ **certain** library calls **may** *synchronize with* other **atomic blocks** **and** library calls performed by another thread.

Add a new paragraph after 6.9.2.1 [intro.races] paragraph 20:

The start and the end of each atomic block (8.8 [stmt.tx]) is a full-expression (6.9.1 [intro.execution]). An atomic block that is not dynamically nested within another atomic block is called an outer block. [Note: Due to syntactic constraints, blocks cannot overlap unless one is nested within the other.] There is a global total order of execution for all outer blocks. If, in that total order, a transaction T1 is ordered before a transaction T2,

- no evaluation in T2 happens before any evaluation in T1 and**
- if T1 and T2 perform conflicting expression evaluations, then the end of T1 synchronizes with the start of T2.**

Two actions are *potentially concurrent* if ...

Change in 6.9.2.1 [intro.races] paragraph 21:

... [Note: It can be shown that programs that correctly use mutexes, **atomic blocks**, and `memory_order::seq_cst` operations to prevent all data races and use no other synchronization operations behave as if the operations executed by their constituent threads were simply interleaved, with each value computation of an object being taken from the last side effect on that object in that interleaving. This is normally referred to as "sequential consistency". ...

Add a new paragraph after 6.9.2.1 [intro.races] paragraph 21:

[Note: The following holds for a data-race-free program: If the start of an atomic block T is sequenced before an evaluation A, A is sequenced before the end of T, and A strongly happens before some evaluation B, then the end of T strongly happens before B. If an evaluation C strongly happens before that evaluation A, then C strongly happens before the start of T. These properties in turn imply that in any simple interleaved (sequentially consistent) execution, the operations of each atomic block appear to be contiguous in the interleaving. -- end note]

Add a production to the grammar in 8.1 [expr.pre]:

```
statement:
    labeled-statement
    attribute-specifier-seqopt expression-statement
    attribute-specifier-seqopt compound-statement
    attribute-specifier-seqopt selection-statement
    attribute-specifier-seqopt iteration-statement
    attribute-specifier-seqopt jump-statement
    declaration-statement
    attribute-specifier-seqopt try-block
    atomic-statement
```

Add a new subclass before 8.8 [stmt.dcl]:

8.8 Atomic statement [stmt.tx]

```
atomic-statement:
    atomic compound-statement
```

An atomic statement is also called an atomic block.

The start of the atomic block is immediately before the opening { of the *compound-statement*.

The end of the atomic block is immediately after the closing } of the *compound-statement*. [

Note: Thus, variables with automatic storage duration declared in the *compound-statement* are destroyed prior to reaching the end of the atomic block; see 8.7 [stmt.jump]. -- end note]

If the execution of an atomic block evaluates any of the following, the behavior is implementation-defined:

- an access through a volatile glvalue;
- an invocation of a function other than a constexpr function with a reachable definition;
- a *throw-expression* (7.6.18 [expr.throw]); or
- a *co_await* expression (7.6.2.3 [expr.await]), a *yield-expression* (7.6.17 [expr.yield]), or a *co_return* statement (8.7.4 [stmt.return.coroutine]).

[Note: Locking a mutex, accessing an atomic variable, or performing I/O invokes non-

constexpr functions.] [Note: The implementation may define that the behavior is undefined in some or all of the cases above.]

A goto or switch statement shall not be used to transfer control into an atomic block.

[Example:

```
int f()
{
    static int i = 0;
    atomic {
        ++i;
        return i;
    }
}
```

```
}  
}
```

Each invocation of f (even when called from several threads simultaneously) retrieves a unique value (ignoring overflow). -- end example]